

Vedak Implementation Package

This document summarizes the software implementations, file organization, programming interface, and build instructions for the Vedak cryptographic hash algorithm. Vedak provides three output-length instances: Vedak-512, Vedak-768, and Vedak-1024. The submission package includes three implementation types: a portable reference implementation, an AVX2-optimized implementation, and an AVX-512 additional implementation. The three implementations are functionally equivalent and produce identical digests for the same instance and the same input message.

1. Directory Structure

The package is organized as follows:

```
API_CryptHash/
├── Implementations/
│   ├── Reference_Implementation/
│   │   ├── Vedak-512/
│   │   ├── Vedak-768/
│   │   └── Vedak-1024/
│   ├── Optimized_Implementation/
│   │   ├── Vedak-512/
│   │   ├── Vedak-768/
│   │   └── Vedak-1024/
│   └── Additional_Implementation/
│       ├── Vedak-512/
│       ├── Vedak-768/
│       └── Vedak-1024/
└── Test_Vector/
    ├── KAT_2_12_Vedak-512.txt
    ├── KAT_2_23_Vedak-512.txt
    ├── KAT_2_33_Vedak-512.txt
    ├── KAT_Loop_Vedak-512.txt
    ├── KAT_2_12_Vedak-768.txt
    ├── KAT_2_23_Vedak-768.txt
    ├── KAT_2_33_Vedak-768.txt
    ├── KAT_Loop_Vedak-768.txt
    ├── KAT_2_12_Vedak-1024.txt
    ├── KAT_2_23_Vedak-1024.txt
    ├── KAT_2_33_Vedak-1024.txt
    └── KAT_Loop_Vedak-1024.txt
```

2. Algorithm Instances

Instance	Digest length	Rate	Capacity	Implementation directories
Vedak-512	512 bits	1536 bits	1024 bits	Reference_Implementation/Vedak-512 Optimized_Implementation/Vedak-512 Additional_Implementation/Vedak-512
Vedak-768	768 bits	1024 bits	1536 bits	Reference_Implementation/Vedak-768 Optimized_Implementation/Vedak-768 Additional_Implementation/Vedak-768
Vedak-1024	1024 bits	512 bits	2048 bits	Reference_Implementation/Vedak-1024 Optimized_Implementation/Vedak-1024 Additional_Implementation/Vedak-1024

Each instance directory is self-contained and can be compiled independently.

3. File Descriptions

File	Description
CryptHash_AlgorithmInstance.c	Implementation of the selected Vedak instance. It defines the CryptHash function together with the internal permutation, padding, absorption, and squeezing procedures.
CryptHash_AlgorithmInstance.h	CryptHash interface header. It defines the algorithm instance name, digest length, optional test-vector output mode, and the prototype of CryptHash.
KAT_CryptHash.c	Known-answer test generator used to produce KAT_2_12, KAT_2_23, KAT_2_33, and KAT_Loop files.
drng.c	Deterministic random-number generator used by the KAT generator.
drng.h	Header file for drng.c.

4. Programming Interface

All implementations provide the same function:

```
int CryptHash(int digest_len_bits,
              const unsigned char *msg,
              unsigned long long msg_len_bits,
              unsigned char *digest);
```

digest_len_bits: requested digest length in bits. It must match the instance in the current directory.

msg: pointer to the input message. For an empty message, this pointer may be NULL only when `msg_len_bits == 0`.

msg_len_bits: input message length in bits. Bit-oriented messages are supported.

digest: output buffer pointer. The buffer size must be at least `digest_len_bits / 8` bytes.

The function returns 0 on success and a non-zero value on error.

5. Build Environment and Compilation

A 64-bit build environment is recommended, especially for generating large KAT files such as KAT_2_33. Typical compilers include GCC, Clang, MSYS2/MinGW-w64 GCC, and Microsoft Visual Studio.

5.1 Reference Implementation

The reference implementation is written in standard C and does not rely on platform-specific vector intrinsics. It is intended mainly for clarity, portability, and independent verification.

Typical GCC command:

```
gcc -O2 -std=c99 -Wall -Wextra \
    KAT_CryptHash.c drng.c CryptHash_AlgorithmInstance.c -o kat
```

On Windows / MSYS2 / MinGW-w64:

```
gcc -O2 -std=c99 -Wall -Wextra \
    KAT_CryptHash.c drng.c CryptHash_AlgorithmInstance.c -o kat.exe
```

5.2 AVX2 Optimized Implementation

The optimized implementation targets mainstream x86-64 processors supporting AVX2. It is intended for most current PC and server environments.

Typical GCC command:

```
gcc -O3 -std=c99 -Wall -Wextra -mavx2 -march=native \
    KAT_CryptHash.c drng.c CryptHash_AlgorithmInstance.c -o kat
```

On Windows / MSYS2 / MinGW-w64:

```
gcc -O3 -std=c99 -Wall -Wextra -mavx2 -march=native \  
KAT_CryptHash.c drng.c CryptHash_AlgorithmInstance.c -o kat.exe
```

5.3 AVX-512 Additional Implementation

The additional implementation targets processors supporting AVX-512. It should be compiled and executed only on platforms that provide the required instruction-set support.

Typical GCC command:

```
gcc -O3 -std=c99 -Wall -Wextra -mavx512f -mavx512bw -mavx512vl \  
-march=native KAT_CryptHash.c drng.c CryptHash_AlgorithmInstance.c -o kat
```

If the target platform does not support AVX-512, use the AVX2 optimized implementation or the reference implementation instead.

6. Note on Warnings in Visual Studio

When the code is compiled with Microsoft Visual Studio, warnings such as C4996 may appear for CRT functions including `fopen`. This does not affect the functional correctness of the algorithm implementation. If needed, the warning can be disabled by adding the following line at the beginning of the source file:

```
#define _CRT_SECURE_NO_WARNINGS
```

This package provides three implementation styles for each Vedak instance. They are functionally equivalent, produce identical test vectors, and differ only in portability and performance.